

实习作业三

1.首先肯定是阅读项目结构和已经有的代码，项目结构不复杂，只有net copy.py和test.py，然后知道了test.py就是对我们搭建训练好的分类模型进行评估分数的，所以后面我们只要注意一下它需要我们输入什么样的模型参数，就可以直接调用了，然后net copy.py就相对复杂很多，里面有注释告诉我们填写自己搭建的CNN。

2.写完CNN以后，我就尝试去运行代码，但是报错内容很多，所以就开始一步步的修改

```
Estimated Total Size (MB): 12.23
=====
标签对应的ID: {'blue': 0, 'red': 1, 'yellow': 2}
Start
Traceback (most recent call last):
  File "/home/yangyuting/桌面/DL/net.py", line 91, in <module>
    for batch_id, (datas, labels) in enumerate(train_loader):
  File "/home/yangyuting/miniconda3/lib/python3.12/site-packages/torch/utils/data/dataloader.py", line 630, in __next__
    data = self._next_data()
           ^^^^^^^^^^^^^^^^^
  File "/home/yangyuting/miniconda3/lib/python3.12/site-packages/torch/utils/data/dataloader.py", line 673, in _next_data
    data = self._dataset_fetcher.fetch(index) # may raise StopIteration
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/yangyuting/miniconda3/lib/python3.12/site-packages/torch/utils/data/_utils/fetch.py", line 52, in fetch
    data = [self.dataset[idx] for idx in possibly_batched_index]
           ~~~~~^~~~~~
  File "/home/yangyuting/miniconda3/lib/python3.12/site-packages/torchvision/datasets/folder.py", line 245, in __getitem__
    sample = self.loader(path)
             ^^^^^^^^^^^^^^^^^
  File "/home/yangyuting/miniconda3/lib/python3.12/site-packages/torchvision/datasets/folder.py", line 284, in default_loader
    return pil_loader(path)
           ^^^^^^^^^^^^^^^^^
  File "/home/yangyuting/miniconda3/lib/python3.12/site-packages/torchvision/datasets/folder.py", line 263, in pil_loader
    img = Image.open(f)
           ^^^^^^^^^^^^^
  File "/home/yangyuting/miniconda3/lib/python3.12/site-packages/PIL/Image.py", line 3498, in open
    raise UnidentifiedImageError(msg)
PIL.UnidentifiedImageError: cannot identify image file <io.BufferedReader name='dataset/train/red/red_104.png'>
```

这里主要原因是有一些图片，它没有办法去读取，所以我们要么加上一个自动调过的代码，要么直接去掉这些图片，我这里是添加了一个自动跳过的代码，现在就完全没有问题了

```

=====
mixed_net          [1, 3]          --
-Conv2d: 1-1        [1, 32, 64, 64]      896
-MaxPool2d: 1-2     [1, 32, 32, 32]    --
-Conv2d: 1-3        [1, 64, 32, 32]    18,496
-MaxPool2d: 1-4     [1, 64, 16, 16]    --
-Conv2d: 1-5        [1, 128, 16, 16]   73,856
-MaxPool2d: 1-6     [1, 128, 8, 8]     --
-Conv2d: 1-7        [1, 256, 8, 8]    295,168
-MaxPool2d: 1-8     [1, 256, 4, 4]    --
-Linear: 1-9        [1, 512]         2,097,664
-Linear: 1-10       [1, 128]         65,664
-Linear: 1-11       [1, 3]          387
=====

Total params: 2,552,131
Trainable params: 2,552,131
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 62.57
=====

Input size (MB): 0.05
Forward/backward pass size (MB): 1.97
Params size (MB): 10.21
Estimated Total Size (MB): 12.23
=====

标签对应的ID: {'blue': 0, 'red': 1, 'yellow': 2}
Start
█

```

3.下一步我是改动输出格式，当前代码的只是：在 epoch>50 且 (epoch+1)%10==0时才打印,然后输出结果也不对，肯定就需要我们去修改一下，修改后输出结果如下所示

```

Train Epoch : 43  Loss : 1.083782
Test -- Average Loss : 1.0797, Accuracy : 44.340
Train Epoch : 44  Loss : 1.082827
Train Epoch : 44  Loss : 1.083546
Test -- Average Loss : 1.0794, Accuracy : 43.396
Train Epoch : 45  Loss : 1.084442
Train Epoch : 45  Loss : 1.083299
Test -- Average Loss : 1.0792, Accuracy : 43.868
Train Epoch : 46  Loss : 1.084069
Train Epoch : 46  Loss : 1.083041
Test -- Average Loss : 1.0789, Accuracy : 44.340
Train Epoch : 47  Loss : 1.086621
Train Epoch : 47  Loss : 1.082757
Test -- Average Loss : 1.0786, Accuracy : 42.925
Train Epoch : 48  Loss : 1.082389
Train Epoch : 48  Loss : 1.082472
Test -- Average Loss : 1.0784, Accuracy : 42.925
Train Epoch : 49  Loss : 1.083183
Train Epoch : 49  Loss : 1.082192
Test -- Average Loss : 1.0781, Accuracy : 41.981
Train Epoch : 50  Loss : 1.079063
Train Epoch : 50  Loss : 1.081878
Test -- Average Loss : 1.0778, Accuracy : 41.981
Train Epoch : 51  Loss : 1.079695
Train Epoch : 51  Loss : 1.081542
Test -- Average Loss : 1.0774, Accuracy : 42.453
Train Epoch : 52  Loss : 1.079031
Train Epoch : 52  Loss : 1.081180

```

4.然后就开始改动 test.py，它的任务一开始就读过，就是把模型加载进来，对两个测试集（test1 和 test2）分别计算总体准确率和每个类别的准确率，然后按作业要求的格式打印。所以我改 test.py 时，重点就是让它能正确读取模型、处理数据、统计并输出。具体来说：需要从 net 中导入

mixed_net 网络结构和 is_valid_image 函数（用来跳过损坏图片，这里是前面提到新增加的）；加载数据集时加上 is_valid_file=is_valid_image 防止坏图报错；模型路径要指向训练时保存的 pth/model_best.pth；然后写一个 evaluate 函数，统计每个类别（蓝、红、黄）的正确数和总数，分别算出总体准确率和类别准确率，按作业的格式打印两遍（test 和 test2）。最后主程序调用两次就行。

```
(base) yangyuting@yangyuting-Yilong15Pro-Series-GM5HG0A:~/桌面/DL$ cd /home/yangyuting/桌面/DL
python3 test.py
/home/yangyuting/桌面/DL/test.py:65: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value)
, which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary co
de during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future r
elease, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpi
ckling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user vi
a `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have f
ull control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  model.load_state_dict(torch.load(model_path, map_location=device))
test Overall Accuracy: 96.698%
Category Accuracy : blue : 100.000%  red : 94.805%  yellow : 94.340%
test2 Overall Accuracy: 87.143%
Category Accuracy : blue : 98.000%  red : 69.388%  yellow : 95.122%
```

然后这就是最终结果，大概就是我做这个的整体思路